

Raytrace: remove repeated Python work

The newest complete Ubuntu session confirms full as the exact optimized default: median runtime falls from 791.2145 to 452.69625 ms per 100 x 100 frame, a 42.78% reduction (1.7478x). The earlier shadow-only change achieved 22.63%. Relative to its historical 611.02 ms optimized median, full is about 25.91% faster; that comparison is across separate sessions, not a direct randomized ablation.

Evidence / candidate	Result	Decision
Ubuntu full, newest session	42.78% lower runtime; exact four-frame gate	Keep the full bundle as default.
Ubuntu shadow_ray, earlier pair	22.63% lower runtime	Keep as the measured intermediate.
Mac individual / slots experiments	Exploratory; not current VM ablations	Retain optional kernels; no isolated VM ranking.
FP64 hardware proposal	Root fraction now measured: 3.864%	Update model; no achieved acceleration.

The benchmark renders a deterministic scene with seven spheres, a halfspace, two point lights, materials, shadows and recursive reflected rays. For each pixel it creates a primary ray, finds intersections, selects the closest valid hit and computes shading/reflections. The exact upstream behavior, including its geometric and checkerboard quirks, is the reference; correcting that behavior would change the benchmark.

Vector, Point, Ray, Sphere, Halfspace and scene/material objects form a graph of ordinary Python objects. Coordinates are boxed Python floats accessed through attributes; object lists drive intersection loops. Canvas stores byte colour data with array. Dependencies are Python math/array and pyperf for the official harness. There is no NumPy or native numeric kernel in the final software.

The baseline wrapper invokes `upstream.bench_raytrace` itself, including its scene setup and render loop. Image-file writing is outside the upstream timer and is enabled only for checks. The optimized variant installs selected methods temporarily and restores them afterward. The original source and baseline wrappers were not changed during this work.

Newest matched pair: `results/raytrace_baseline_20260920-074314_1800` and `results/raytrace_optimized_20260920-075734_1800`. Comparison: `results/session_all_20260920-074308_674/comparison_raytrace`. Session explicitly selects full; all captured source hashes match the reviewed files.

Measure the workload before changing it

The target is Ubuntu 22.04.5, Linux 5.15.0-1106-kvm, one virtual CPU exposing an Intel Xeon E5-2630 v3 at 2.40 GHz. Clean timing uses release CPython 3.10.12, CPU 0 affinity, a fixed hash seed and cyclic GC disabled. The VM governor is unavailable; host scheduling, frequency and contention are not fully controlled. These are measurements of this guest, not bare-metal Haswell limits.

Instrument	What it establishes	What it cannot establish
Clean timing	11 fresh processes per arm; upstream internal timer; every observation retained.	No proof against between-block drift.
perf stat	Separate repeated counting runs; instructions, cycles, branches and loads per equal work.	Its elapsed time is not the clean timing result.
perf record + FlameGraph	Native debug stacks, fixed 5-million-cycle sampling period; aggregate all folded stacks.	Not a release-time percentage or a count of Python calls.
cProfile	Exact call counts and Python function attribution in a separate run.	Tracing time is perturbed and is not a speedup measurement.
pyperformance / pyperf	Official suite baseline cross-check in its worker harness.	No official optimized-suite result is present.

The custom wrapper calls the actual upstream bench function and uses its timer. A minimal pyperf import stub exposes `time.perf_counter`; it does not implement `pyperf.Runner`. Thus this is an upstream-workload experiment through a custom harness, alongside a separate official pyperformance baseline run. That distinction does not invalidate the experiment; the workload, timer boundaries and comparison controls must be explicit.

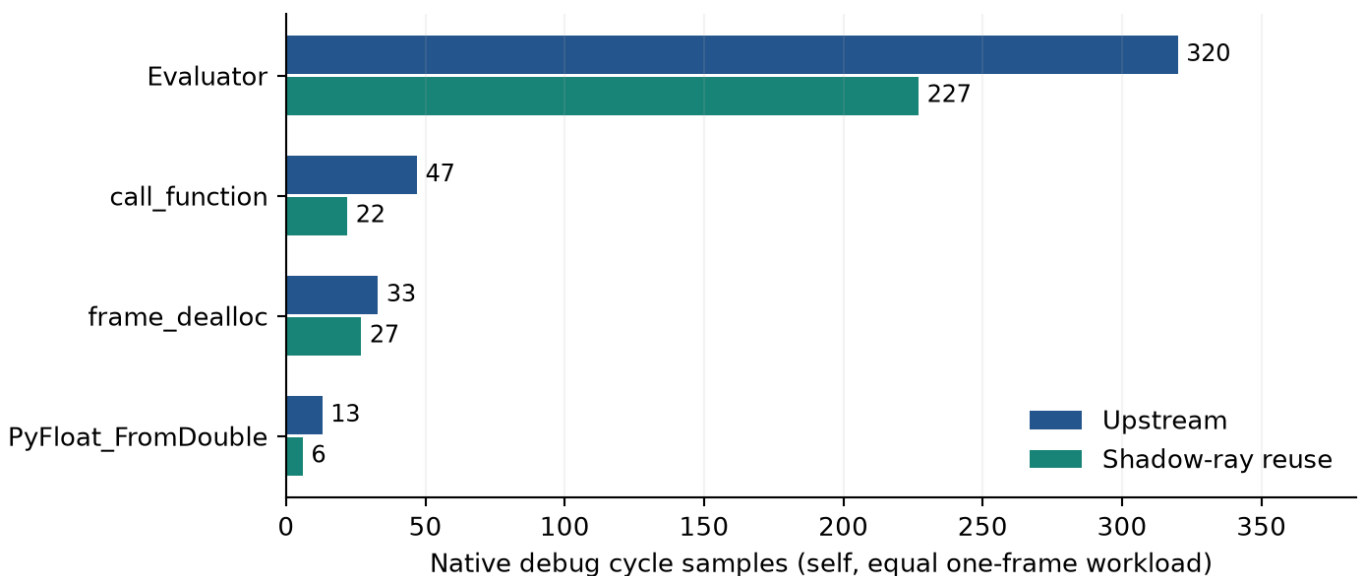
Runtime reduction = $100 \times (1 - \text{optimized median} / \text{baseline median})$. Speedup = $\text{baseline median} / \text{optimized median}$. We use runtime reduction for the declared 7% target. CV = $\text{sample standard deviation} / \text{mean}$. Samples from separate TXT and CSV perf invocations are not pooled.

Latest session saves a four-frame exact optimized gate covering both reference and candidate. Standalone baseline verification is smaller despite its phase label; the saved optimized gate provides the actual batch comparison. Some perf phases remain labelled requested although their artifacts exist. `py-spy` is explicitly disabled; official pyperformance applies only to the baseline.

Initial hypothesis and historical shadow evidence

A flame graph is an aggregation of sampled call stacks, not a timeline. Width is inclusive sampled weight; height is stack depth, not elapsed time. Self weight counts only samples whose leaf is that symbol. Children weight includes descendants and overlaps other ancestors. The libc start routine can therefore have a huge inclusive share while doing little direct work. Aggregate a function across every folded stack: the top-30 listing is incomplete, and repeated appearances must not be mistaken for separate causes.

The repeating evaluator/vectorcall chain reflects nested Python method execution. Tall similar stacks alone cannot count calls or price each call. cProfile supplies the stronger mechanism: one frame has 509,873 dot calls, 277,865 Point.__sub__/isPoint calls and 179,457 sphere intersection calls. In total it records 3,092,298 calls, including non-Python calls. A fixed claim of 1,600 machine instructions for every call is not supported.



Equal work: one frame per native debug profile, period 5 million cycles. These selected rows are self counts. Total samples: 1,073 baseline and 823 shadow-ray. Counts are indicative; sparse differences are not precise estimates.

The actionable question is whether identical geometry is repeatedly wrapped in new objects. In `lightIsVisible`, the same origin and light direction are used while testing successive objects. Reconstructing and normalizing that ray for every object repeats work without changing the ray. Reusing it once per visibility query predicts fewer Ray/Vector creations, normalizations, dot products and guard calls, while leaving the number of intersection queries unchanged.

CPython is a bytecode interpreter, not an AST interpreter. `_PyEval_EvalFrameDefault` includes bytecode execution, operand handling, loop control and reference work; its self share is not pure call overhead. Debug checks and different code generation change costs. High IPC and few cache misses support an instruction-work hypothesis but do not identify a precise hardware stall limit. Reject negative top-down fractions and unsupported PMU events rather than interpreting them as valid measurements or zeros.

Shadow reuse: the measured first change

`shadow_ray` builds one ray per visibility query and reuses it for each object test. This requires read-only intersection methods; custom or mutating implementations use the original path. It retains object traversal order, short-circuit behavior, comparison thresholds and upstream handling of intersections beyond the light. It does not introduce a different visibility rule.

One-frame cProfile count	Upstream	<code>shadow_ray</code>	Interpretation
Ray construction	98,172	26,000	72,172 duplicate constructions removed.
Vector construction	452,955	308,611	144,344 fewer objects; import overhead in both.
normalization / magnitude	109,887	37,715	Same removed ray-normalization work.
dot	509,873	437,701	Arithmetic helper calls decrease.
Sphere intersection	179,457	179,457	Same geometric query count.
<code>lightsVisible</code>	10,666	10,666	Same visibility queries.
All recorded calls	3,092,298	2,299,196	25.65% fewer calls; not a runtime fraction.

This combination is stronger evidence than a large interpreter percentage: a concrete source change removes a known repeated computation; exact call counts change by the predicted amount; equal-work instructions and clean runtime both fall. It supports redundant Python work as an important cost, without claiming a single universal bottleneck.

Pixel identity was checked at 24 x 24, 100 x 100 and 37 x 23. The 100 x 100 PPM SHA-256 prefix is 11f2c36b02044f11. Current verification checks every frame in the requested batch, independent sizes, and baseline restoration. A matching image alone is supplemented by raw colour, geometry and randomized tests for the new full kernel.

Historical shadow artifacts do not fully capture the optimized source hash, and manifests record a dirty working tree. The workload records and saved tests support the result, but do not reconstruct every historical local edit. Current workflows capture source hashes and explicit session paths.

The full exact kernel and the decision ledger

Candidate	Change / reason	Status
guards	Stock-class fast paths avoid polymorphic calls; preserve original fallback for other objects.	Included; alone +1.31% time locally.
sphere_scalar	Inline scalar dot/subtraction arithmetic; remove temporary objects; retain expression order.	Included; alone -14.64% locally.
camera	Reuse column/row camera components without reordering arithmetic.	Included; alone -1.07%.
nearest_hit	Direct selection instead of intermediate hit list/tuples; retain evaluations, ties and EPSILON.	Included; alone -2.14%.
checkerboard	Remove discarded Vector result while preserving reciprocal and scalar arithmetic/exceptions.	Included; alone +1.25%.
full	Combine those candidates with shadow-ray reuse.	Default; -38.55% locally.
slots / full_slots	Fixed attribute layout replaces stock geometry classes; different object-model contract.	Optional; full_slots -39.81%.

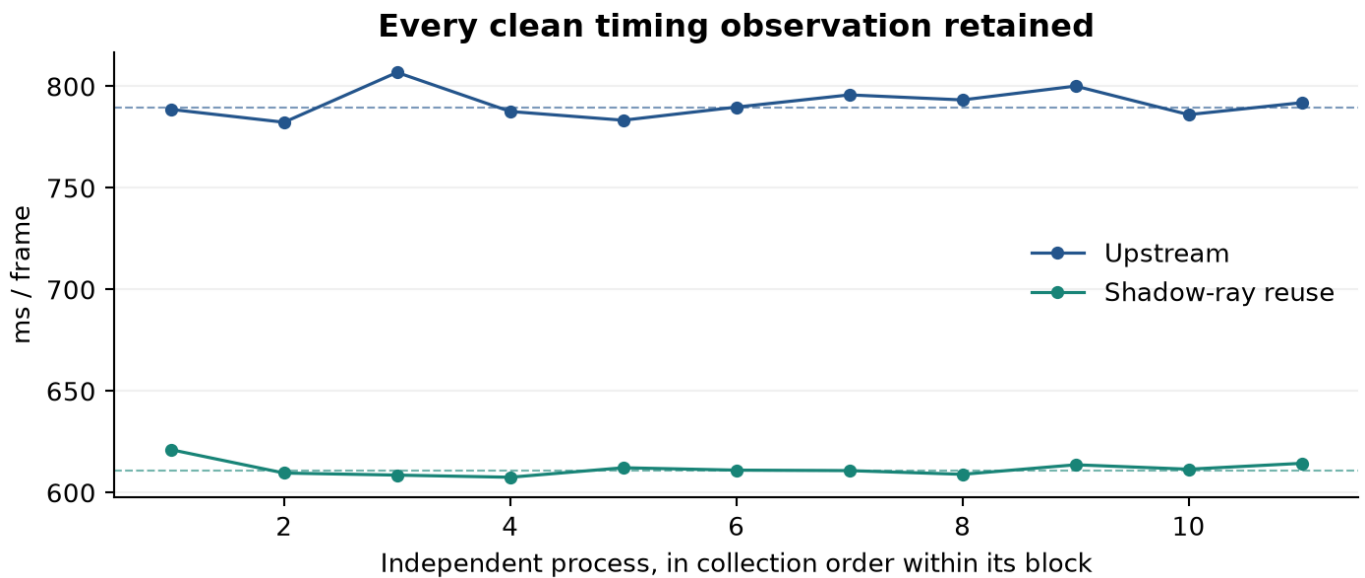
These local results use CPython 3.14.6 on arm64 macOS, 11 rotating repetitions and two 100 x 100 frames per sample. They are exploratory, not Ubuntu measurements; small individual regressions are not resolved reliably. Effects are not additive, and a faster combination does not prove every component helps. Keep explicit kernels for target-VM ablation.

Strict correctness means bit-identical benchmark pixels and preserved tested computation at successful boundaries. Tests include raw float bits for 1,000 randomized sphere cases and edge cases, camera components, prequantization colours, varied scenes, custom fallbacks, ties and restoration. All four production-sized frames match in the saved full-batch gate. Twenty-four raytrace tests passed; all 63 combined software tests passed.

Rejected or deferred: unconditional guard deletion breaks polymorphism; exact-class rejection without fallback narrows behavior; fixing upstream shadow/checkerboard semantics changes the task. Lower precision, approximate roots, a new integrator or answer caching violate the exact reference contract. Compiled native backends remain a possible separate experiment, outside the agreed pure-Python implementation.

Retention decision after the new Ubuntu run: keep full as a tested bundle. The additional work reductions occur in the predicted functions and the whole bundle is much faster. No new VM isolated or leave-one-out ablation was collected, so this does not prove every component contributes positive runtime savings. In particular, guards and checkerboard were slightly slower alone in the old Mac experiment. Keep those controls and optional slots available; do not claim their individual VM gains.

Historical shadow-only runtime comparison



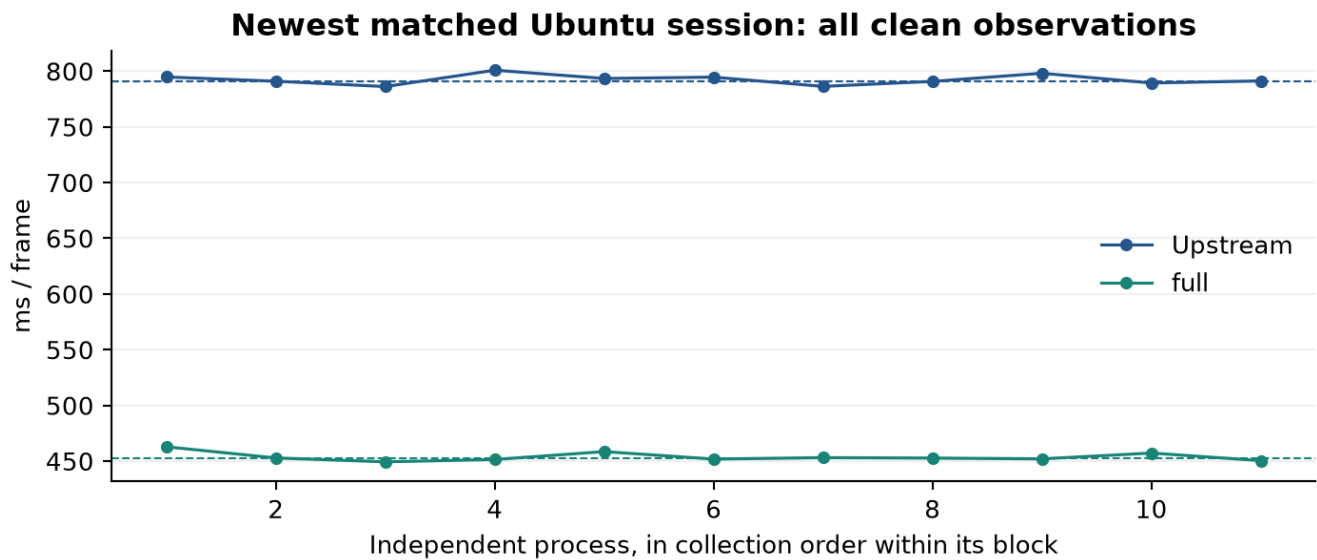
Eleven independent processes per arm, four frames per process. Dashed lines are medians. Every observation is retained; arms were collected in separate blocks.

Metric, normalized per frame	Upstream	shadow_ray	Change
Median clean runtime	789.756 ms	611.023 ms	-22.63% / 1.2925x
Mean +/- sample SD	791.416 +/- 7.344 ms	611.757 +/- 3.745 ms	CV 0.93% / 0.61%
Instructions (perf TXT)	5.068 billion	3.892 billion	-23.20%
Cycles (perf TXT)	1.915 billion	1.487 billion	-22.38%
Branches / L1 data loads	653.0m / 1,639.1m	497.2m / 1,263.4m	-23.85% / -22.92%

Runtime, instructions, cycles and calls move together, as predicted by removal of repeated setup. A separate perf CSV batch also shows about 23% fewer instructions; do not merge it with TXT samples. L1 misses decrease 13.38%, yet their rate rises from 2.65% to 2.98% because loads fall faster. A worse rate does not mean more misses or a slower program.

All optimized timing observations beat all baseline observations. The effect is much larger than within-block variation, although VM drift remains unmeasured. The official pyperformance baseline reports about 800 +/- 5 ms, consistent with the custom baseline. It is a cross-check, not an official optimized result.

Newest full run: a further substantial improvement



Eleven independent processes per arm, four frames per process; all observations retained. Dashed lines mark medians. Arms were collected in separate blocks.

Clean timing per unit	Upstream	Optimized
Median	791.214500 ms	452.696250 ms
Mean +/- sample SD	792.408 +/- 4.550 ms	453.830 +/- 4.022 ms
CV	0.574%	0.886%
Minimum - maximum	786.148 - 800.841 ms	449.354 - 462.805 ms

Median runtime reduction is 42.7846%, saving 338.51825 ms/frame. Even the slowest optimized process beats the fastest baseline by 41.13%. Dispersion is small relative to the effect, but sequential collection leaves slow VM drift unmeasured. The generated 2x-noise rule is a heuristic, not a significance test.

The earlier shadow-only baseline was 789.7555 ms, only 0.185% below this baseline; its optimized median was 611.02275 ms. Full is 25.91% below that older optimized median. This is corroborating historical evidence, not an isolated estimate of the added components because the experiments were collected at different times.

Official baseline cross-check: 20 value-producing workers, 60 values; median about 801 ms, mean 801 +/- 5 ms. It uses a different harness and does not supply an official optimized result. Full source SHA: 5badaf27f0f9519fd0d2779bd37c745feadcf76e31fbde833cd89dfbb5770d55.

What changed: fewer calls, objects and instructions

One-frame calls	Upstream	shadow_ray*	full
Sphere intersections	179,457	179,457	179,457
Ray construction	98,172	26,000	26,000
Vector construction	452,955	308,611	105,497
dot	509,873	437,701	78,787
Point subtraction	277,865	205,693	26,236
mustBeVector	520,543	448,371	4
Nearest-hit helper	15,265	15,265	0
Intersection list comprehension	15,265	15,265	0
Normalization	109,887	37,715	37,715

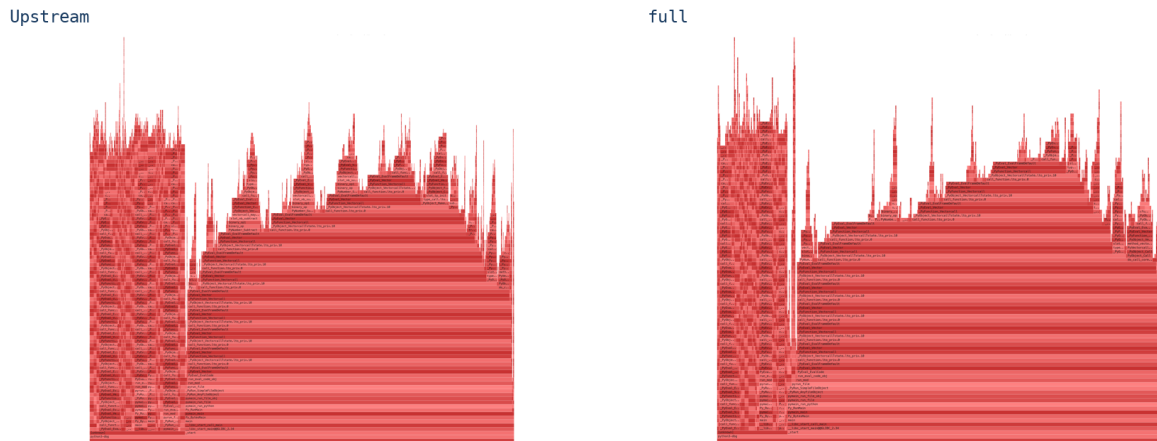
* Shadow column is the historical shadow-only profile. The deterministic query counts match across scenes; this is a work comparison, not a same-session runtime ablation. New full total process calls: 3,092,309 -> 881,684 (-71.49%), including interpreter/library calls.

All 179,457 sphere tests remain. Vector constructions fall by 347,458: shadow reuse removes 144,344; scalar spheres 179,457; camera reuse 19,800; checkerboard 3,857. These sum exactly to the observed delta. Scalar spheres also remove two dot helpers per test. Direct hit selection removes the intersection list/helper. Exact images and unchanged primitive, visibility and pixel counts support preserved work; these counts do not isolate each change's runtime contribution.

Counter / unit (TXT)	Upstream	Optimized	Change
Instructions	5,050.629m	2,843.416m	-43.70%
Cycles	1,900.076m	1,104.609m	-41.87%
L1 data loads	1,639.573m	915.284m	-44.18%
Branches	650.184m	368.754m	-43.28%

A separate five-run CSV counter batch corroborates the TXT result: instructions -43.87%, cycles -42.41%, L1 loads -43.75%. Both batches show about 43% fewer L1 load misses. IPC decreases modestly (TXT 2.658 -> 2.574); fewer instructions explain the speedup. These are separate counting experiments and include process startup outside the clean timer.

Similar stack shapes; markedly less sampled work



Original native flame-graph overviews, each independently normalized. Original SVGs are retained for zooming. The one-frame profiles use debug CPython and a fixed 5-million-cycle period.

Equal-work native observation	Before -> full	Interpretation
All native samples	1,061 -> 552	47.97% fewer sampled cycles; not the clean runtime reduction.
Evaluator self	263 -> 174	Absolute decrease; share rises 24.79% -> 31.52%.
call_function self	45 -> 22	Consistent with fewer Python calls.
frame_dealloc self	34 -> 14	Less frame-lifetime work.
Attribute/dict self group	184 -> 87	Defined symbol group; not a complete disjoint partition.

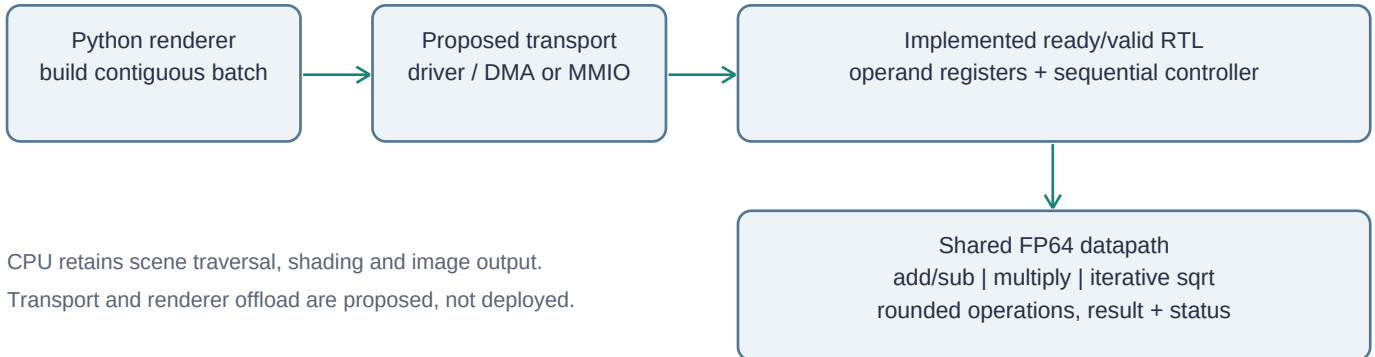
The interpreter and renderer remain on the stack, and both SVGs fill their canvas width. The remaining evaluator share can grow while its absolute samples fall. The normalized differential graph shows redistribution, not saved seconds; wrapper ancestry also changes. Unknown frames occur somewhere in 240 baseline and 88 optimized sample paths, so exact attribution remains incomplete.

The remaining largest traced Python function is scalar sphere: 0.29546 s self and 0.29646 s cumulative across 179,457 calls. Its self time increased after work was inlined, while inclusive sphere time fell from about 0.78140 s. That is attribution moving into the function, not a regression. ray_colour, visibility, shading and remaining vector helpers follow. Tracing times are not release fractions or hardware speedup ceilings.

L1 miss rate rises slightly (2.966% -> 3.031%) despite fewer misses. Generic cache-miss counts disagree across TXT/CSV; cache-sampling runs have only 12 and 3 samples. Reject cache hotspot claims from them. Negative top-down fractions are invalid; LLC events are unsupported. Kernel/BPF warnings qualify symbol interpretation; reported lost samples are zero.

A small, explicit FP64 accelerator

The proposal offloads a complete ray-sphere intersection rather than individual floating-point instructions. This boundary can remove Python objects, calls and dispatch as well as execute arithmetic.



Implemented accelerator boundary is the ready/valid RTL. The host transport is an integration proposal.

hardware/rtl/ray_sphere_accel.sv contains the sequential controller, operand/result registers, status and ready/valid protocol. The datapath uses actual unmodified BSD-licensed Berkeley HardFloat Release 1 RTL: one shared add/sub unit, one multiplier and an iterative square-root unit. There are no real-number or software-math placeholders in the synthesizable datapath.

```

cp = center - origin
v = (cp.x*dx + cp.y*dy) + cp.z*dz
q = (cp.x*cp.x + cp.y*cp.y) + cp.z*cp.z
disc = r*r - (q - v*v)
result = no_root if disc < 0 else v - sqrt(disc)
  
```

Each operation rounds separately to binary64, nearest-even, including gradual underflow. There is no FMA, reassociation, FP32 or approximate reciprocal root. A tangent and a negative returned distance preserve upstream behavior; software applies its original hit-selection rule. Finite inputs/intermediates, nonnegative radius and caller-normalized direction define the supported domain. Invalid/nonfinite cases return an error for CPU fallback.

HardFloat source and license: hardware/vendor/ provenance and license files. Primary specification: <https://www.jhauser.us/arithmetic/HardFloat-1/doc/HardFloat-Verilog.html>. Detailed ports, schedule and host format: docs/HARDWARE_DESIGN.md and hardware/.

Design reassessment: sphere intersections remain frequent, but square root is reached only 6,934 times (3.864% of 179,457 queries). The current core is a single-query sequential function with no resident scene or internal batch controller. A batched geometric core or a small reusable FP64 command engine are alternatives, not implemented replacements. More general hardware is not automatically faster; operand residency and the software offload boundary need justification.

Resource sharing limits replication and potential switching but also throughput. FP64 rounding/sqrt remain substantial logic; no numerical area, power or achieved frequency is claimed. A pipelined or multi-lane version trades area/control/clocking for throughput. An equivalent native CPU batch is necessary to separate escaping Python from hardware value.

New workload evidence changes the estimate

Interface / validation	Current specification
Request / response	640 operand bits + 32-bit tag; distance/status/flags response; one outstanding query; ready/valid and held response.
Numerical contract	Separate FP64 rounding, ordered expressions, caller-normalized rays; finite supported domain with CPU fallback on error.
Implementation boundary	Arithmetic/control RTL and host packing exist. Renderer batching, transport/DMA and driver remain proposed.
Clock / schedule	50 MHz target assumption; 20 cycles/miss, 80/root budget. No achieved timing or HDL simulation.

The full-profile math.sqrt caller map records 6,934 sphere calls; 37,715 other square roots belong to magnitude and one to import setup. Thus $h = 6,934 / 179,457 = 0.0386388$. Source control flow confirms that sphere calls sqrt only after a nonnegative discriminant. This measured workload mix replaces the earlier all-root assumption; it is not a measured hardware speed.

With the same assumed 50 MHz, 1 GB/s aggregate payload bandwidth, batch size 1,024, 10 us launch/batch and 100 ns native packing/query, modeled core time is 80.1036 ms. Transfer is 18.6635 ms, launch 1.7600 ms and packing 17.9457 ms: 118.4728 ms total offload cost. Against the new full software time of 452.69625 ms, break-even requires removing more than 26.17% of release runtime.

Assumed removed fraction f	Estimated total time	Reduction vs full
20%	480.630 ms	-6.17% (slower)
30%	435.360 ms	3.83%
40%	390.091 ms	13.83%

$T_{\text{new}} = T_{\text{full}} \times (1 - f) + T_{\text{offload}}$. f remains unknown: cProfile sphere share cannot supply it. The modeled 7% runtime reduction would require $f > 33.17\%$; this is a sensitivity point, not a separate mandated hardware target. Batching can change early-exit queries and h, so the original query mix must be rechecked after integration.

The old all-root case against shadow-only software remains historical evidence, not the current default estimate. If applied to faster full software, that all-root stress case needs 71.90% removable time just to break even. No hardware improvement occurred between estimates; only software baseline and workload assumptions changed.

What is established and what comes next

Full is now the selected exact raytrace implementation, with a 42.78% target-VM runtime reduction and a saved four-frame correctness gate. The earlier 22.63% shadow result remains the first measured step. Additional scalarization, call removal and direct traversal visibly remove work; the aggregate result does not establish the isolated value of every component.

Keep all diagnostic kernels, including optional slots, for controlled comparisons. New individual VM ablations are needed before ranking or pruning small changes. Hardware remains a candidate: the measured low root fraction improves the analytical estimate, while faster software tightens the budget. The next hardware decision should compare a batched geometric core with a small reusable arithmetic engine and account for native CPU execution.

```
python3 -B variants/bm_raytrace_upstream_opt.py \
--mode verify --loops 4
# Exploratory within-process ablation; all kernels, rotating order:
python3 -B variants/bm_raytrace_upstream_opt.py --mode ablate \
--loops 4 --reps 22 --no-gc --json raytrace_ablation.json
# Confirm a selected bundle with independent-process paired rounds:
LOOPS=4 ./tools/ab_timing.sh raytrace 16 --kernel full
```

This session records exact baseline/optimized paths and source hashes without Git commands. The kernel, upstream, wrappers, common/pipeline and configuration hashes match current files. The old UNVERIFIED provenance text and absent authenticity transcript remain limitations: hashes identify the local source but do not alone certify package origin. Original results and latest-pointer links were not modified by this review.

Submission component	Location / status
Required six-section TXT report	report_raytrace.txt, exported from the same content as this PDF.
Scripts and README	script_raytrace.sh, run_all.sh, README.md.
Code / validation / experiments	variants/, tests/, docs/evidence/, docs/RAYTRACE_FULL_EXPERIMENT.md.
Profiles and raw results	Selected results/ directories; docs/reports/evidence_registry.json; original flame SVGs in assets/.
Hardware architecture and HDL	hardware/ and docs/HARDWARE_DESIGN.md; limitations stated above.
AI prompts / presentation	prompt.txt records actual prompts. Presentation (20-25 min plus questions), live demo and repository upload remain separate actions.

Course source: Project.pdf, pp. 3-5 and 7-8; requirement mapping in docs/PROJECT_REQUIREMENTS.md. No Git operations or repository publication were performed in this work.